

Choosing between efficiency & performance with RocksDB

Mark Callaghan

Member of Technical Staff, Facebook

RocksDB, MyRocks & MongoRocks

RocksDB

- Embedded key-value storage engine

MyRocks

- RocksDB storage engine for MySQL
- coming to Percona Server and MariaDB Server

MongoRocks

- RocksDB storage engine for MongoDB
- In Percona Server today

RocksDB, MyRocks & MongoRocks

Efficient performance is the goal

- Good read efficiency
- Better write efficiency
- Best space efficiency

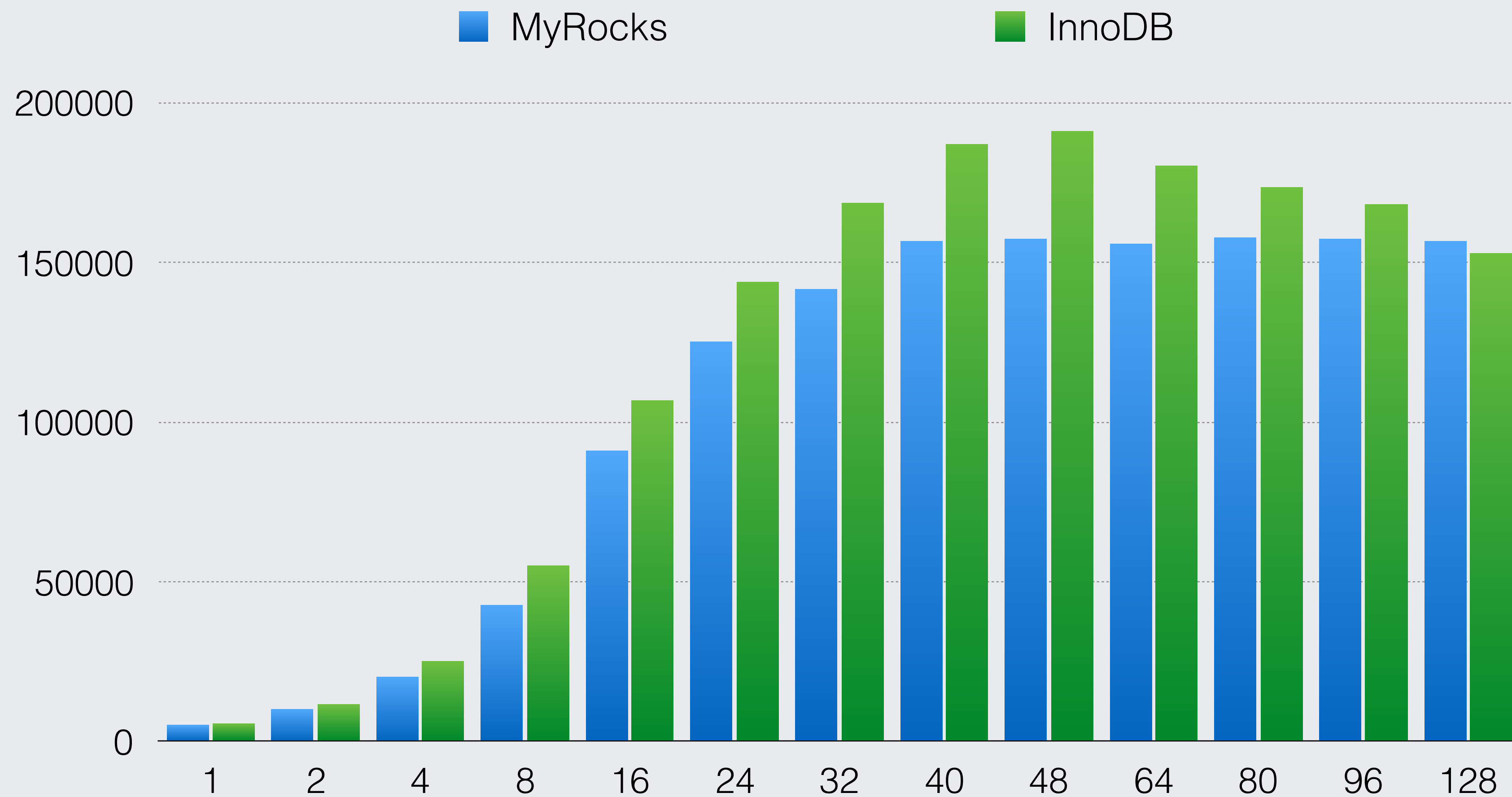


- Use less SSD
- Use lower endurance SSD

In some cases, better read efficiency is possible

Benchmarking?

Sysbench read-write, in-memory



Better is not just about throughput

- QPS - average throughput
- QoS - worst case throughput
- Efficiency - hardware per query

	tpmC	iostat rKB/t	iostat wKB/t	vmstat CPU/t	Size (GB)	p99 response (ms)
MyRocks +zlib	95680	2.19	2.02	528	82	29.9
InnoDB	91981	2.67	7.49	400	222	18.4

tpcc-mysql, 1000 warehouses, IO-bound

Peak performance is overrated

Performance



Performance & Efficiency

Meet performance goals

Then optimize for efficiency

RUM Conjecture

RUM or RWS?

- RUM = **R**ead, **U**ppdate, **M**emory
- RWS = **R**ead, **W**rite, **S**pace

Efficiency

- Synonym for amplification
- Related to, but not equivalent to, performance.

An algorithm can't be optimal for all of read, write & space amplification

- See *Designing Access Methods: The RUM Conjecture*
- daslab.seas.harvard.edu/rum-conjecture

Define: optimal

For any useful database algorithm there exists another useful algorithm that has better read, write or space amplification.

Define: amplification

Read, Write

- physical work per logical request

Space

- $\text{sizeof}(\text{database files}) / \text{sizeof}(\text{data})$

Define: work

Storage

- random operations
- bytes read
- bytes written

CPU

- blocks (un)compressed
- memory/cache operations
- hash searches
- key comparisons
- CPU seconds

Define: operation

Basic operations:

- point read
- range read
- put
- delete

Complex operations:

- query
- transaction

Update is one of:

- put
- point read, put
- point read, delete, put

Everything is relative

	tpmC	iostat rKB/t	iostat wKB/t	vmstat CPU/t	Size (GB)
MyRocks+zlib	95680	2.19	2.02	528	82
InnoDB	91981	2.67	7.49	400	222
InnoDB/ MyRocks	0.96	1.22	3.71	0.76	2.71

tpcc-mysql, 1000 warehouses, IO-bound

Efficiency: B-Tree

Example: InnoDB

Read

- $\log N$ key compares

Write

- $\text{sizeof}(\text{page}) / \text{sizeof}(\text{row})$

Space

- 1.5X if leaf pages are 2/3 full

Update-in-Place B-Tree

- InnoDB

Copy-on-Write B-Tree

- WiredTiger
- LMDB

Efficiency: leveled LSM

Example: RocksDB, Cassandra

Read

- $\log N + \log(N/10) + \log(N/100) + \log(N/1000)$ key compares
- point reads can use bloom filter

Write

- rewrite previously written rows
- worse than size-tiered LSM, better than B-Tree

Space

- 1.1X

Efficiency: size-tiered LSM

Example: RocksDB, Cassandra, HBase

Read

- more than leveled LSM
- point reads can use bloom filter

Write

- rewrite previously written rows
- better than leveled LSM, better than B-Tree

Space

- ~2X, worse than leveled LSM

Efficiency: summary

	Read	Write	Space
B-Tree	best		good
leveled LSM	good for point	good	best
size-tiered LSM		best	

Theory meets practice

IO costs are hard to predict

Access distribution

- LSM benefits from skew

Cache

- B-Tree - prefer to have index in cache
- LSM - prefer to have all but largest level in cache

Linkbench, IO-bound

MyRocks: best throughput & QoS, most efficient

	TPS	iostat r/t	iostat wKB/t	CPU usecs/t	Size (GB)	p99 update (ms)
MyRocks+zlib	28965	1.03	1.25	999	374	1
InnoDB	21474	1.16	19.70	914	14xx	6
InnoDB+zlib	20734	1.07	14.59	1199	880	6

Why did RocksDB beat a B-Tree?

Space efficiency

- Fragmentation
- Fixed page size
- More per-row metadata
- No prefix encoding (InnoDB)

Write efficiency

- Uses more space = more data to write
- Working set larger than cache
- $\text{sizeof}(\text{page}) / \text{sizeof}(\text{row})$
- Double write buffer (InnoDB)

Page size & write amplification

	Page size	TPS	iostat wKB/t	
MyRocks+zlib	16kb	28965	1.25	
	InnoDB	4kb	24845	6.13
	InnoDB	8kb	24352	10.52
	InnoDB	16kb	21414	19.70

Read efficiency: B-Tree vs LSM?

Performance is complex

Advantage B-Tree

- Fewer key comparisons
- Less IO for range queries

Advantage LSM

- Uses less space = more data in cache
- Prefix key encoding when uncompressed
- Efficient writes saves IO for reads
- Read-free index maintenance
- Bloom filter

Save on writes, spend more on reads

	MyRocks zlib TPS	InnoDB TPS	Ratio (MyRocks / InnoDB)
Disk array	2195	414	5.3
Slow SSD	23484	10143	2.3
Fast SSD	28965	21414	1.4

Trading between R, W and S efficiency

Read versus Write efficiency

- Indexes - more & wider

Read versus Space efficiency

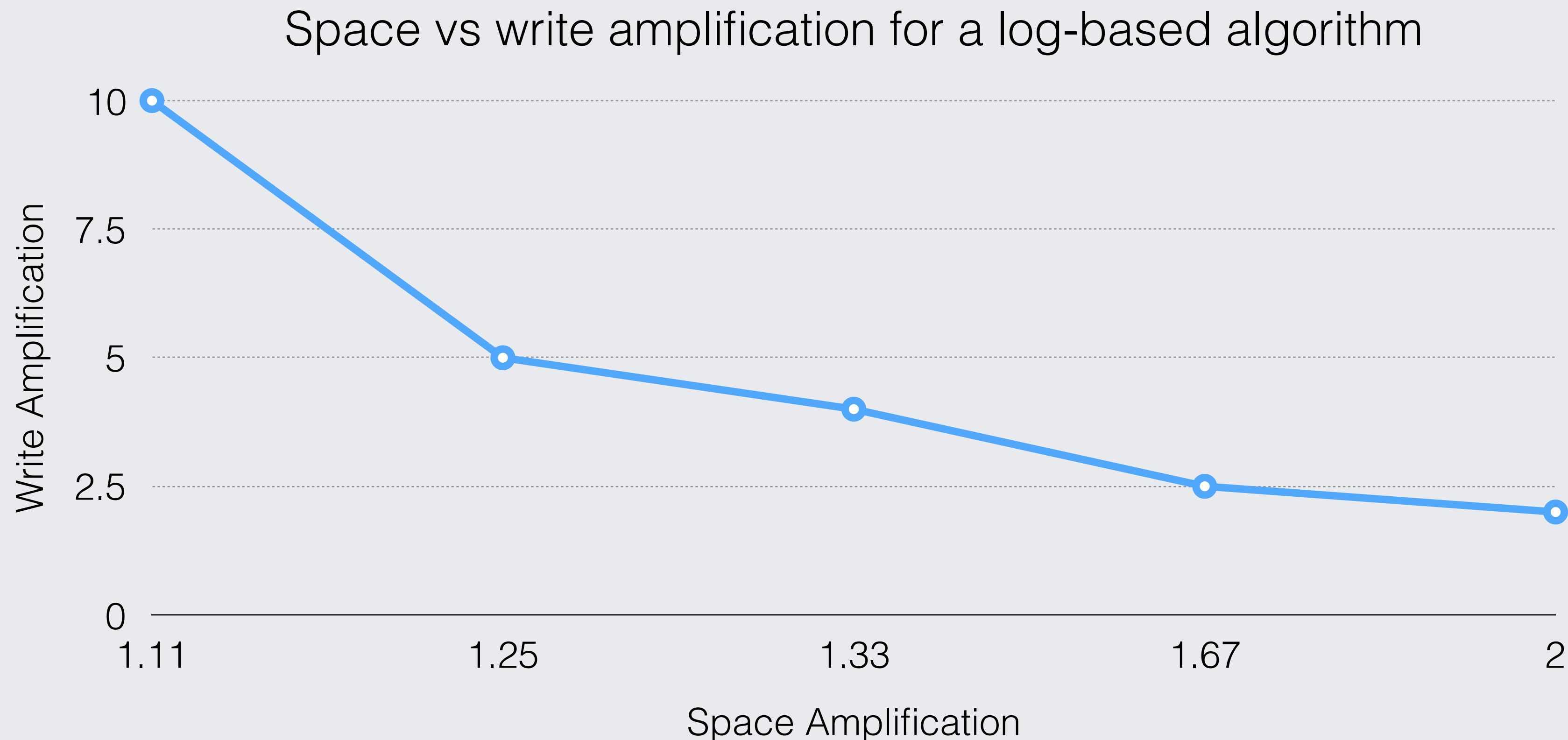
- Bloom filters
- Compression
- Indexes

Write versus Space efficiency

- RocksDB fanout
- Size-tiered vs leveled
- GC & defragmentation frequency

Write versus space amplification

- space amplification = $100 / \%full$
- write amplification = $100 / (100 - \%full)$



What comes next?

One size doesn't fit all

- B-Tree + LSM sharing one redo log

Adaptive algorithms

- DBA sets high-level goals
- Algorithm adapts to achieve them

More open source

- MyRocks in MariaDB Server & Percona Server
- MongoRocks in Percona Server
- More features in RocksDB

Thank you

rocksdb.org

mongorocks.org

github.com/facebook/mysql-5.6

smalldatum.blogspot.com

twitter.com/markcallaghan

